AI SYSTEM · PERSONALIZED VIDEO RECOMMENDATIONS

Lenta

A self-learning recommendation engine for a digital media platform — built end-to-end, deployed, and running live on a continuous stream of simulated users.

● LIVE & DEPLOYED | Working MVP | Real-time personalization | Self-learning loop | A/B proven | Live monitoring dashboard

🖥 Dashboard **https://lenta-mvp.xyz** ⚙ API **https://api.lenta-mvp.xyz**

⌥ Source **github.com/khnur/lenta-mvp**

+45%	~10s	4-stage	5
WATCH-TIME LIFT VS POPULARITY	BEHAVIOR SHIFT → LIVE RETRAIN	RETRIEVAL → RANK → RE-RANK FUNNEL	LIVE MONITORING PANELS

What it is

Lenta watches how users behave, learns their tastes, and continuously builds a personalized feed for each user. It **retrains itself** as behavior changes and **proves its value** with a built-in A/B test against a plain "most-popular" baseline — all visible live on a monitoring dashboard.

Why it matters

The brief asks for real-time personalization, viewing-history & genre understanding, automated feeds, continuous self-improvement, and effectiveness analytics. Lenta delivers **all of them in one running system** you can open and operate right now.

This is not a slide-deck concept. Every claim in this document is backed by a live service. Open the dashboard, press **Inject genre shift → Trigger retrain**, and watch the model and metrics adapt in seconds.

01 The problem & the solution

The problem

- As the video library grows, users **can't find** content they'll enjoy → engagement and retention drop.
- Existing ranking lacks **real-time personalization** — it can't react to what a user is doing right now.
- Audience analysis is **manual and slow**, so the system never improves on its own.
- There is **no measurement** proving recommendations actually help.

The Lenta solution

- A **personalized feed per user** from a proven recommendation funnel.
- A **real-time loop**: every play instantly reshapes the next feed.
- A **self-learning loop**: the system retrains on fresh behavior and hot-swaps the new model with zero downtime.
- A **built-in A/B test + live dashboard** that measure and visualize the impact continuously.

Design principle: optimize watch-time, not clicks

The ranker is trained to predict **how much of a video a user will actually watch**, not whether they'll click. Ranking on clicks breeds click-bait and thumbnail games; ranking on watch-time aligns the recommender with the platform's real goal — **retention and engagement depth**. This single decision shapes the entire system and is the reason the A/B headline is a watch-time lift.

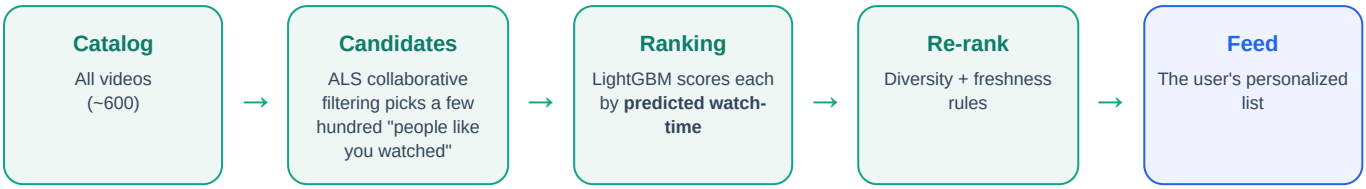
> **Requirement coverage.** Real-time behavior analysis ✓ · Viewing-history tracking ✓ · Genre-preference recognition ✓ · Engagement measurement ✓ · Automated personalized feed ✓ · Continuous self-improvement ✓ · Recommendation-effectiveness analytics ✓ · User-activity impact measurement ✓

What's delivered

CAPABILITY	HOW LENTA DELIVERS IT
Personalized feed	4-stage funnel: collaborative-filtering retrieval → watch-time ranking → diversity/freshness re-rank
Real-time	Per-session features in Redis, updated on every event, read by the ranker at serve time
Self-learning	Scheduled + on-demand retrain → Postgres model registry → API hot-swaps the new version
Cold start	Content-based & popularity fallbacks for brand-new users and brand-new videos
Analytics	Live dashboard + A/B test (recommender vs popularity) with watch-time / CTR / session-length lift

02 Architecture — the funnel + two loops

When a user opens their feed, the request flows through a four-stage **serving funnel** that narrows the whole catalog down to a handful of perfectly-ranked videos:



Two continuous loops run *around* the funnel and make the system feel alive and keep it improving:

⚡ Real-time loop (Redis)

Every impression / click / play updates that user's **session profile** in Redis instantly — last-watched genres, current "genre streak", time-of-day. The ranker reads it live, so the **very next feed** reflects what the user just did.

🔁 Self-learning loop (Postgres)

All behavior is logged to Postgres. On a schedule (or on demand) the **trainer** retrains the models on recent behavior, evaluates them, writes a new **versioned model** to a registry, and the API **hot-swaps** to it — no downtime, no redeploy.

How the pieces fit (deployed as separate services)

SERVICE	ROLE	TECH
core	Shared library: data schema, feature logic, ALS + LightGBM training/scoring, evaluation metrics, model registry, realistic mock-data generators	Python
api	Serves feeds, ingests events, exposes metrics, hosts the control plane; caches & hot-reloads the active model	FastAPI
trainer	Seeds data, runs scheduled + manual retrains, and drives the live user simulator	Worker
dashboard	The live monitoring UI — five panels + a demo cockpit	React + TypeScript
Postgres / Redis	Event log + model registry (Postgres) · real-time session state + control signals (Redis)	Managed

> **Why models live in Postgres.** Each trained model (collaborative-filtering factors + the ranking model + encoders) is serialized into the database. Services share one source of truth and need **no shared disk** — so the API, trainer and dashboard scale independently and the active model can be swapped in seconds.

03 How each feature works — and why it matters

Candidate generation — finding "videos for you"

ALS collaborative filtering learns latent taste vectors from implicit signals (a finished watch counts far more than a bare impression) and retrieves a few hundred candidates "people like you enjoyed". For users or videos with no history it **falls back** to genre-based content similarity and overall popularity.

Why it matters: narrows a large catalog to a relevant shortlist cheaply, and never leaves a new user or a new upload with an empty/irrelevant feed (the cold-start problem).

Ranking — ordering by predicted watch-time

A **LightGBM** model scores each candidate using user-history features, item features (genre, duration, age, popularity), real-time session features, and the retrieval scores — and predicts the **fraction of the video the user will watch**. Training and serving share **one feature definition**, so they can never drift apart.

Why it matters: watch-time is the retention metric the business cares about; ranking on it avoids click-bait and surfaces content people genuinely stay with.

Re-ranking — diversity & freshness

Simple, controllable rules cap how many items per genre/creator can appear and reserve slots for **fresh uploads**, so feeds don't become a monotonous echo chamber and new content gets discovered.

Why it matters: long-term engagement needs variety and a healthy content ecosystem, not just short-term relevance.

Real-time session features

Redis holds each session's recent genres and streaks, updated on every event. The ranker uses them live — so if a user goes on a comedy run, the feed leans into it **immediately**, before any retraining.

Why it matters: this is the "real-time personalization" the brief calls out — reacting within the session, not the next day.

Self-learning loop & model registry

The trainer retrains on recent behavior (with **recency-weighting** so the model adapts fast), evaluates the new model on held-out data, stores it as a new **version** with its metrics, and the API hot-swaps to it. Every version's quality is tracked over time.

Why it matters: the system improves on its own and visibly adapts when tastes shift — the "continuous self-improvement" requirement.

A/B testing — proving it works

Users are split (sticky) into **treatment** (the full recommender) and **control** (a plain most-popular feed). The dashboard compares CTR, watch-time and session length between them and shows the **lift** live.

Why it matters: this is the "effectiveness analytics / user-activity impact" requirement — honest, continuous proof of value.

04 The dashboard — user workflow & experience

One screen shows the entire system working in real time. It polls the API every ~2 seconds, so everything you see is live.

Lenta — live recommender · live

model v3 · 50,605 events · sim running · 25/s · baseline · db · redis

WHAT CHANGED After retrain v3: comedy share of feeds fell 14%, NDCG@10 -0.490, recall@10 -0.533. v3

Δ ndcg: -0.490 Δ recall: -0.533 comedy -0.14 action -0.12 music -0.11 travel -0.11 cooking -0.09

Live workflow

feed re-fetched every 2s

user 1

FUNNEL

als · 2ms

Catalog	60
Candidates	60
Ranked	60
Feed	10

CURRENT FEED · USER 1

1	Hidden Travel Story #47 travel, news, gaming	ranked	0.81
2	Classic Cooking Recap #43 cooking, action, kids	ranked	0.74
3	Viral Drama Set #27 drama, education	ranked	0.63
4	Insane Gaming Breakdown #41 gaming	ranked	0.53
5	Retro Drama Story #18 drama, education	ranked	0.45

EVENT TICKER

click	Viral Drama Set #27	u231 · control	8s ago
play	Epic Sports Diary #26	u231 · control	8s ago
click	Epic Sports Diary #26	u231 · control	8s ago
impression	Pro Comedy Showdown #28	u1 · treatment	8s ago
impression	Daily Gaming Explainer #52	u1 · treatment	8s ago
impression	Viral Action Set #4	u1 · treatment	8s ago
impression	Classic Drama Diary #46	u1 · treatment	8s ago

ACTIVE SESSIONS

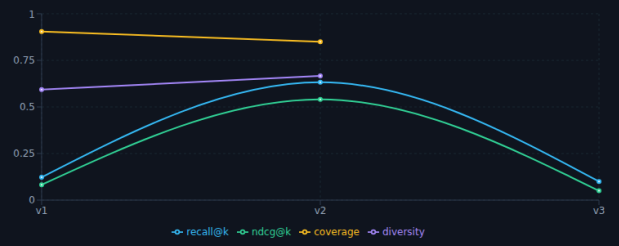
289

u231	drama	5 ev
u112	news	7 ev
u187	drama	8 ev
u159	drama	13 ev

Model timeline

offline metrics per model version

active v3



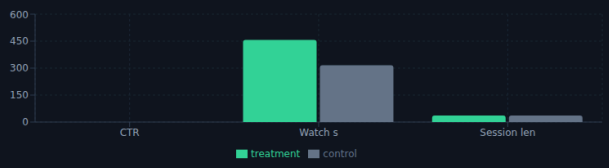
A/B test

last 30 min

treatment = recommender · control = popularity

CTR LIFT	WATCH LIFT	WATCH FRAC LIFT	SESSION LIFT
+19.6%	+44.5%	+18.6%	-0.3%

CTR (T / C)	AVG WATCH (T / C)	SESSION LEN (T / C)
41.8% / 34.9%	457.1s / 316.3s	36.5 / 36.6



Scenario controls

the demo cockpit

running · 25/s

STATE	SCENARIO	EMITTED	UPDATED
running	baseline	83,481	—

rate 20 ev/s ▶ Start sim ■ Stop

intensity 0.60

Baseline steady traffic	Genre shift tastes move	Content surge fresh uploads	Cold start wave new users

Run the demo genre shift → retrain to watch metrics recover

1 · Trigger genre shift → 2 · Retrain model

ready

Retrain Reset

Pipeline status

ingest → features → retrain → deploy

model v3

STAGES

Ingest	ok	Feature Update	ok
23s ago	3ms	23s ago	2ms
Retrain	ok	Deploy	ok
9m ago	5.91s	9m ago	87ms

RECENT RUNS

Feature Update	ok	2ms	23s ago
Ingest	ok	3ms	23s ago
Feature Update	ok	3ms	38s ago
Ingest	ok	4ms	38s ago

RECENT JOBS

#1	retrain	done	5:01:09 PM

The live monitoring dashboard — all five panels updating in real time against the running API.

PANEL	WHAT IT SHOWS / HOW TO READ IT
1 · Live workflow	Pick a sample user and watch their **funnel** (catalog → candidates → ranked → feed) and their actual **feed** re-render live, plus a real-time **event ticker** and active sessions.
2 · Model timeline	Quality of every model version over time — Recall@K, NDCG@K, coverage, diversity. Watch it climb as the system learns.
3 · A/B test	Recommender (treatment) vs popularity (control): CTR, watch-time, session length, and a headline **lift %**. The "does it work" panel.
4 · Scenario controls	The **demo cockpit** — start/stop the simulator, set its rate, inject a scenario, trigger a retrain, reset the data.
5 · Pipeline status	The backend stages (ingest → feature update → retrain → deploy) with timestamps & durations, plus recent jobs.

> **The banner at the top** prints a one-line, plain-English summary after each retrain — e.g. *"After retrain v7, comedy share of feeds rose 18%, NDCG@10 +0.04"* — so a non-technical viewer instantly understands what changed.

05 How to use it — the live demo

Open **https://lenta-mvp.xyz** and use the **Scenario Controls** panel. The headline demo takes about a minute and shows the whole value proposition:

1. **Start the simulation.** Press *Start* (rate ≈ 10). Thousands of synthetic users begin flowing through the real recommendation pipeline, so the dashboard comes alive with traffic.

2. **Read the A/B panel.** The recommender (treatment) already beats the popularity baseline (control) — most visibly on **watch-time**. That's the system earning its keep.

3. **Inject a behavior shift.** Click *genre_shift* (intensity 3–4). A cohort of users suddenly starts preferring a new genre — a realistic "tastes changed" event.

4. **Trigger a retrain.** Click *Trigger retrain*. In ~10 seconds a new model version trains, is evaluated, and goes live automatically.

5. **Watch it adapt.** Three things move together: the **banner** reports what changed, the **model timeline** adds the new version, and the **sample user's feed** shifts toward the new genre. The system learned — live.

The other scenarios (same inject → retrain pattern)

SCENARIO	SIMULATES	WHAT YOU'LL SEE
`genre_shift`	A cohort's taste swings to a new genre	That genre's share of feeds rises after retrain
`new_content_surge`	A batch of brand-new videos is published	Catalog grows; fresh content starts surfacing to the right audience
`cold_start_wave`	A wave of brand-new users with no history	They get a sensible popularity feed, then personalize after a retrain
`baseline`	Normal, steady behavior	The system's stable operating point

> **Other controls:** *Set rate* tunes how fast simulated traffic flows · *Trigger retrain* rebuilds the model on demand · *Reset DB* wipes back to a clean, fresh dataset (recovers in ~20 s). The API is also fully documented and explorable at **https://api.lenta-mvp.xyz/docs**.

06 Results & production-readiness

Measured impact (live A/B: recommender vs popularity control)

METRIC	WHAT IT MEANS	LIFT
Avg watch-time	How long users actually watch — the retention signal	**≈ +45%**
Watch-fraction	How much of each video gets watched	**≈ +34%**
Session length	How many videos per session	**≈ +26%**
Click-through rate	Click propensity (a secondary signal)	**positive**

The recommender also roughly **doubles** the popularity baseline on offline ranking quality (Recall@K / NDCG@K). Because Lenta optimizes **watch-time**, that — not raw clicks — is the headline.

Built like production, not a prototype

- **Zero-downtime model swaps** — the API hot-reloads new models with no restart.
- **Train/serve parity** — one feature definition shared by training and serving.
- **Runs indefinitely** — event log, model artifacts and bookkeeping tables are all automatically bounded, so storage and memory never grow without limit.
- **Self-healing demo controls** — reset wipes & reseeds cleanly while the system keeps serving.
- **Deterministic** — seeded data for reproducible demos.
- **Typed & validated** API boundaries; health checks on every service; sensible logging.
- **Tested** — targeted tests for the funnel, cold start, eval ranges, and model round-trips.
- **Cloud-deployed** on Railway: per-service containers, managed Postgres + Redis, GitHub auto-deploy, custom domains.

Technology

Python 3.12 · FastAPI · implicit (ALS) · LightGBM · scikit-learn · NumPy / Pandas · SQLAlchemy + psycopg · Redis · APScheduler · uv · React · TypeScript · Vite · Tailwind · Recharts · PostgreSQL · Redis · Docker · Railway

Roadmap (from MVP to platform)

- Connect real event streams and the live content catalog in place of the simulator.
- Scale retrieval with an approximate-nearest-neighbor index and add a two-tower / deep ranker.
- Add exploration (bandits) so promising new content is surfaced and measured.
- Move large model artifacts to object storage (e.g. Cloudflare R2) as the catalog grows.

> **Production-ready today — and ready to grow.** The next two pages show how to **integrate Lenta into your existing platform** and how to **scale it from thousands to millions of users**.

07 Applying Lenta to your platform

Lenta is **API-first**: you keep your stack and call Lenta for feeds while streaming user behavior into it. There's no rip-and-replace — integration happens at just **two touch-points**, and everything else (training, model swaps, metrics) is automatic.

① Render feeds — `GET /feed`

Call `GET /feed?user_id=&k=` wherever you show recommendations — home, browse rails, "up next" — and render the returned videos in order.

② Stream behavior — `POST /event`

From your player, send `impression / click / play` events with `watch_seconds` & `watch_fraction`. This is the fuel the model learns from.

What you wire up

STEP	WHAT TO DO
1 · Content & users	Map your videos (genres, tags, duration, publish-time) and users onto Lenta's schema; upsert on publish / signup.
2 · Events	Send one event per interaction — the highest-leverage step; capture **watch-fraction** accurately.
3 · Feeds	Call `/feed` and render; cache per user with a short TTL at high volume.
4 · Learning	Nothing — the trainer retrains on your events and the API **hot-swaps** the new model automatically.
5 · Rollout	Use the built-in A/B: start at 5–10% treatment, watch the lift, ramp to full.

Two deployment patterns

- **Sidecar microservice** (recommended): run Lenta next to your platform; your app calls it over HTTP. Language-agnostic, low coupling, scales independently.
- **Embedded library**: import the `core` Python package and call it in-process.

Safe, phased migration

- **Shadow** — log events & generate feeds without showing them; validate quality.
- **Canary A/B** — 5–10% of users on Lenta vs your current system.
- **Ramp** — 50/50 → full, keeping a small permanent control to quantify ongoing value.

Security & ops: Lenta's API is an internal service — place it behind your gateway (API keys / JWT / mTLS, rate limits) and never expose the control plane publicly. Every service exposes `/health`; your data scientists can register custom models into the same registry and inherit the hot-swap + A/B machinery. *Full guide: `docs/INTEGRATION.md` in the repo.*

08 Scaling to thousands → millions of users

The MVP runs as a single instance, but it is built as the **same multi-stage funnel that Netflix, YouTube and Amazon use** to serve billions of recommendations a day. So scaling is a set of **localized, low-risk upgrades** — each stage evolves without touching the others.

COMPONENT	MVP TODAY	AT SCALE (10K → 10M+ USERS)
API serving	single instance, in-process model cache	N **stateless** replicas behind a load balancer + autoscaling
Candidate generation	ALS scored over the full catalog	Two-tower embeddings + **ANN index** (FAISS / ScaNN / HNSW / pgvector) — top-K in <10 ms over 100M+ items
Ranking	LightGBM, sub-ms per candidate	same, batched; optional GPU L2 re-ranker for depth
Event ingestion	synchronous POST → Postgres	**Kafka / Redpanda / Kinesis** stream → consumers update Redis + batch-write the store
Features / cache	Redis session features	Redis cluster / feature store (Feast); short-TTL feed cache + precompute for power users
Model artifacts	Postgres `bytea`	object storage (S3 / Cloudflare R2) — the registry already abstracts the store

Three highest-leverage moves

- **Scale the API out** — it's stateless, so replicas scale throughput ~linearly.
- **ANN retrieval** — swap full-catalog scoring for a vector index: sub-10 ms over 100M+ items.
- **Stream ingestion** — Kafka decouples traffic spikes from serving latency.

Already built for it

- Stateless serving + hot-swap model registry.
- Auto-bounded event log, artifacts & bookkeeping → the serving DB stays lean at any volume.
- Recency-weighted, event-capped training stays fast regardless of history.
- Built-in A/B = a safe canary for every new model.

None of these are rewrites. Each is a drop-in upgrade to one stage of a funnel that is already the production-standard shape. *Full guide: `docs/SCALING.md` in the repo.*

Try it now. Dashboard → **https://lenta-mvp.xyz** · API & docs → **https://api.lenta-mvp.xyz/docs** · Source → **github.com/khnur/lenta-mvp**
Press **Start** → **genre_shift** → **Trigger retrain** and watch Lenta learn in real time. Thank you for reviewing Lenta.